Ders 6 Çalışma Özeti

Ders 6 Çalışma Özeti

Genel Konular

- Bu kayıt uygulama oturumudur.
  - İlk 8 dakika bekleme kısmıdır. Asıl oturum yaklaşık 00:08'de başlar.
  - Uygulamayı asistan Toprak Hoca yaptı.
- Uygulama sorusu: Kitaplık uygulaması (`Uyg1.pdf`)
  - Üç sınıf: `Kitap`, `Raf`, `Kitaplik`. Bir de `Test` sınıfı.
  - Soru UML diyagramıyla verildi. Kod bu diyagrama göre NetBeans'te yazıldı.
- Konular
  - `private`, `final` ve `static` alanlar. Statik sayaç (`kitapSayisi`) ve statik metot (`isbnNedir`).
  - `toString()` ile nesneyi anlamlı yazdırma.
  - Nesne dizisi oluşturma ve dizideki boş (`null`) yeri bulma.
  - Gelişmiş `for` (for-each) ile iç içe dizilerde arama.
  - == ile nesne karşılaştırma.
  - UML'de bileşim (composition, dolu elmas) ve birleştirme (aggregation, boş elmas) ilişkileri.

Hocanın Özellikle Vurguladığı Kısımlar

- **Algoritmalar basit tutulacak.** Amaç nesne yönelimli kavramları anlamak.
  - Önemli olan hangi sınıfların olduğu ve sınıflar arasındaki ilişkilerdir.
- **UML'den kod yazmayı ve soru metninden UML çıkarmayı bilin.**
- **Bileşim (composition) ve birleştirme (aggregation) farkı önemli.**
  - Kitaplık yok olursa rafları da yok olur (bileşim, dolu elmas). Raflar `Kitaplik` constructor'ı içinde üretilir.
  - Raf yok olsa da kitaplar var olmaya devam eder (birleştirme, boş elmas). Kitaplar dışarıda üretilip rafa eklenir.
  - Örnekler: İnsan yok olursa beyni de yok olur. Araba yok olsa da yolcu var olur.
  - Toprak Hoca'nın akılda tutma yolu: bileşim "ondan oluşur", birleştirme "onu içerir".
- **Metodu ilgili sınıfa yazın.**
  - `Kitap` sınıfına `kitapEkle` yazmak mantıksızdır. Kitabın ekleneceği yer raf veya kitaplıktır.
  - Hangi sınıfta hangi alan ve metot olacağını çıkarmak bu işin tasarım tarafıdır. Örnek çözdükçe gelişir.
- **Sorgu metodu nesne döndürebilir.**
  - `kitapAra` bir `Kitap` nesnesi döndürür. Online alışverişte bir ürünün adını yazınca bütün bilgileri gelir. Arka plandaki mantık budur.
  - Aynı adda birden fazla kitap varsa bir kitap dizisi de döndürülebilir.
- **Kapsülleme sayesinde iç yapı değişebilir.** Raf sınıfını kullanan kişi `kitapEkle` ve `kitapSil`'in nasıl çalıştığını bilmez. Veritabanı değişse bile kullanan kod aynı kalır.
- Anlamadığınız her şeyi sorun. Konuyu YouTube'da da bulabilirsiniz; bu saatlerin değeri sorduğunuz sorulardadır.

Kısa Tekrar Notları

- UML: - private, + public. `kitapSayisi: <static> int` statik alandır.

- Public sınıfın adı dosya adıyla aynı olmalı. Değiştirmek için NetBeans'te Refactor > Rename kullanılır.
- `final` alan (ISBN) yalnız bir kez atanır. Setter'ı olmaz, yalnız getter'ı olur.
- Statik sayaç constructor içinde artırılır: `kitapSayisi++`.
- Statik metot sınıf adıyla çağrılır: `Kitap.isbnNedir()`. `Kitap.getAd()` çağrılamaz, çünkü hangi kitabın adı olduğu belli değildir.
- `System.out.println(k)` aslında `k.toString()` çağırır.
- `new Kitap[10]` on kitap üretmez; on tane `null` referans içeren bir dizi üretir.
- `new Raf[n]` sonrasında her eleman ayrıca `new Raf()` ile üretilmelidir. Yoksa `NullPointerException` alınır.
- Dizide boş yer arama: `while (i < dizi.length && dizi[i] != null) i++;`
- Silme: ilgili hücreye `null` atanır.
- String karşılaştırma `equals` ile yapılır. Önce `null` kontrolü yapılır.
- == iki referans aynı nesneyi gösteriyorsa `true` döner. İçerikleri aynı iki farklı nesne için `false` döner.

Detaylı Açıklamalar

1. Soru ve UML

Ekran karesinden alınan UML (`Uyg1.pdf`):

- `Kitap`: `- ad: String`, `- yazar: String`, `- ISBN: String`, `- kitapSayisi: <static> int`. Metotlar: `Kitap(String,String,String)`, `getAd()`, `getYazar()`, `getISBN()`, `getKitapSayisi(): int`, `setAd(String)`, `setYazar(String)`, `toString()`.
- `Raf`: `- kitaplar: Kitap[]`. Metotlar: `Raf()`, `kitapEkle(Kitap)`, `kitapSilSiraNo(int)`, `kitapSil(Kitap)`, `rafGoster()`, `getKitaplar(): Kitap[]`.
- `Kitaplik`: `- raf: Raf[]`. Metotlar: `Kitaplik(int)`, `kitapEkle(Kitap,int)`, `kitapSilSiraNo(int,int)`, `kitapSil(int,Kitap)`, `kitapAra(String): Kitap`, `gosterRaf(int)`, `tumRaflariGoster()`.
- İlişkiler: `Kitaplik` dolu elmasla `Raf`'a bağlı (bileşim). `Raf` boş elmasla `Kitap`'a bağlı (birleştirme). `Test` iki sınıfı kullanır (kesikli ok, bağımlılık).

2. Kitap sınıfı

```
public class Kitap {
    private String ad;
    private String yazar;
    private final String ISBN;          // bir kez atanır
    private static int kitapSayisi = 0; // bütün kitaplar için ortak

    public Kitap(String ad, String yazar, String ISBN) {
        this.ad = ad;
        this.yazar = yazar;
        this.ISBN = ISBN;
        kitapSayisi++;
    }

    public String getAd() { return ad; }
    public String getYazar() { return yazar; }
    public String getISBN() { return ISBN; }
    public static int getKitapSayisi() { return kitapSayisi; }
    public void setAd(String ad) { this.ad = ad; }
    public void setYazar(String yazar) { this.yazar = yazar; }

    public static String isbnNedir() {
        return "isbn kitaplari ayirt etmek icin essiz bir numaradir";
    }

    @Override
```

```
    public String toString() {
        return "Kitap(Ad=" + ad + ", Yazar=" + yazar + ", ISBN=" + ISBN + ")";
    }
}
```

- NetBeans'te Insert Code ile constructor, getter/setter ve `toString` otomatik üretildi.
- `ISBN` `final` olduğu için NetBeans setter üretmedi.
- Test:

```
System.out.println(Kitap.isbnNedir());
Kitap kitap1 = new Kitap("Kitap1", "Yazar1", "1234");
System.out.println(kitap1);        // Kitap(Ad=Kitap1, Yazar=Yazar1, ISBN=1234)
kitap1.setAd("Kitap2");
System.out.println(kitap1);        // Kitap(Ad=Kitap2, ...)
System.out.println(Kitap.getKitapSayisi());  // oluşturulan kitap sayısı
```

- Bir öğrenci statiğin amacını sordu. Hocanın cevabı:
  - Statik metot: Sık kullanılan, nesne gerektirmeyen işler için. Sınıf adıyla çağrılır.
  - Statik alan: Sınıfın bütün nesneleri için ortak bilgi. Örnek: o ana kadar kaç kitap üretildiği. Her nesnede baştan başlamaz, kaldığı yerden devam eder. Bellekte farklı bir bölgede tutulur.

3. Raf sınıfı

Kod ekran karesinden alındı:

```
public class Raf {
    private Kitap[] kitaplar;

    public Raf() {
        this.kitaplar = new Kitap[10];   // 10 boş (null) yer
    }

    public void kitapEkle(Kitap k) {
        int i = 0;
        while (i < kitaplar.length && kitaplar[i] != null)
            i++;
        if (i != kitaplar.length)        // boş yer bulunduysa
            kitaplar[i] = k;
    }

    public void kitapSilSiraNo(int siraNo) {
        if (siraNo < kitaplar.length)
            kitaplar[siraNo] = null;
    }

    public void kitapSil(Kitap k) {
        int i = 0;
        while (i < kitaplar.length && kitaplar[i] != k)
            i++;
        if (i != kitaplar.length)
            kitaplar[i] = null;
    }

    public void rafGoster() {
        for (int i = 0; i < kitaplar.length; i++)
            System.out.println(kitaplar[i]);   // boş yerlerde "null" yazar
    }

    public Kitap[] getKitaplar() { return kitaplar; }
}
```

- `while` şartındaki sıra önemlidir. Önce `i < kitaplar.length` kontrol edilir. Böylece dizinin dışına taşılmaz.
- `kitapSil(Kitap k)` içindeki `kitaplar[i] != k` karşılaştırması referans karşılaştırmasıdır. Aynı nesne aranır.

4. Kitaplik sınıfı

```
public class Kitaplik {
    private Raf[] raf;

    public Kitaplik(int n) {
        this.raf = new Raf[n];
        for (int i = 0; i < raf.length; i++)
            raf[i] = new Raf();          // raflar kitaplığın içinde üretilir (bileşim)
    }

    public void kitapEkle(Kitap k, int rafNo)          { raf[rafNo].kitapEkle(k); }
    public void kitapSilSiraNo(int rafNo, int siraNo)  { raf[rafNo].kitapSilSiraNo(siraNo); }
    public void kitapSil(int rafNo, Kitap k)           { raf[rafNo].kitapSil(k); }
    public void rafGoster(int n)                       { raf[n].rafGoster(); }

    public void tumRaflariGoster() {
        for (int i = 0; i < raf.length; i++) {
            System.out.println(i + ". Raftaki Kitaplar");
            rafGoster(i);
        }
    }

    public Kitap kitapAra(String kitapAdi) {
        int n = raf.length;
        for (int i = 0; i < n; i++)
            for (Kitap kitap : raf[i].getKitaplar())
                if (kitap != null && kitap.getAd().equals(kitapAdi)) {
                    System.out.println(i + ". Rafta Kitap");
                    return kitap;
                }
        System.out.println("Kitap yok");
        return null;
    }
}
```

- `Kitaplik` metotlarının çoğu işi `Raf` sınıfına devreder. Raf sınıfını başka biri yazmış olsa da kitaplık kodu değişmez.
- `kitapAra` içindeki for-each:
  - Dış döngü rafları dolaşır.
  - İç döngü o rafın `getKitaplar()` dizisini dolaşır. Her adımda `kitap` değişkeni sıradaki elemanı gösterir.
  - `raf[i].kitaplar` alanına doğrudan erişilemez, çünkü `private`'tır. Bu yüzden getter kullanıldı.
  - Boş hücrelerde `kitap.getAd()` çağrısı `NullPointerException` verir. `kitap != null` kontrolü bunu önler.
- Toprak Hoca bu metodu bilerek kitaplığa yazdı. Daha doğru tasarım: `Raf` sınıfına bir `kitapAra` metodu yazmak ve kitaplıktan onu raf raf çağırmak.

5. Test

```
Kitaplik kitaplik = new Kitaplik(3);
kitaplik.kitapEkle(kitap1, 0);
kitaplik.kitapEkle(kitap2, 0);
kitaplik.kitapEkle(kitap3, 0);
```

```
kitaplik.kitapEkle(kitap4, 1);
kitaplik.kitapEkle(kitap5, 1);
kitaplik.kitapEkle(kitap6, 2);
kitaplik.kitapSilSiraNo(0, 1);   // 0. raftaki ortadaki kitap silinir
kitaplik.kitapSilSiraNo(1, 1);   // 1. rafta kitap4 kalır (00:49:32)
kitaplik.tumRaflariGoster();
```

- Hocanın denediği durum: `kitapSil(0, kitap6)`. 6. kitap 0. rafta olmadığı için kod hata vermez, hiçbir şey yapmaz. Ama `rafNo` geçersizse `ArrayIndexOutOfBoundsException` olur, çünkü `raf[rafNo]` kontrol edilmiyor.

6. == ile nesne karşılaştırma

- `kitap1 == kitap1` -> `true`.
- Adı, yazarı ve numarası aynı iki farklı nesne için `==` -> `false`. İçerik karşılaştırması için `equals` ezilmelidir.

7. Öğrencilerin tasarım önerileri

- Rafa tek kitap yerine bir kitap dizisi ekleme. Hoca bunun dinamik dizilerle (ArrayList) daha kolay olacağını, ileride işleneceğini söyledi.
- Kitap silinince sonraki kitapları sola kaydırma (shift). Statik dizide bu da yapılabilir.

8. Sonraki haftalarda

- İlerleyen haftalarda `ArrayList` ve nesne sıralama (object sort) işlenecek. İki kitap nesnesinin yer değiştirmesi orada tekrar görülecek.
- İlişki türlerinden association ve extend (kalıtım) gelecek haftalarda işlenecek.

Ders sonrası notlar

- **UML'deki `toString(): void` hatası:** Soru dosyasındaki UML'de `toString(): void` yazıyor. `toString` her zaman `String` döndürür: `+ toString(): String`. Kodda da öyle yazıldı.
- **UML'de statik gösterimi:** Diyagramda `kitapSayisi: <static> int` yazılmış. Standart UML'de statik üyelerin altı çizilir. `getKitapSayisi()` da statik olduğu için onun da altı çizilmelidir.
- **`kitapAra` dönüş değeri:** Ekran karesindeki ilk sürümde döngülerden sonra `return` yoktu (NetBeans satırı hatalı gösteriyordu). Kitap bulunamazsa da bir değer döndürülmeli: `return null;`. Yukarıdaki kodda bu eklendi. Çağıran taraf da sonucu `null` için kontrol etmeli.
- **Bileşim ve çöp toplayıcı:** "Kitaplığı silersem raflar da silinir" ifadesi Java için şu anlama gelir: Kitaplık nesnesine referans kalmazsa ve raflara da başka referans yoksa, çöp toplayıcı ikisini de toplar. Dışarıda `getRaf` ile bir rafın referansı tutuluyorsa o raf yaşamaya devam eder. Java'da C++'taki gibi açık `delete` yoktur.
- **Yazım farkı:** Koddaki metot adları UML ile tam aynı değil (`rafGoster(int)` - UML'de `gosterRaf(int)`; `KitapSilSiraNo` büyük harfle başlıyor). UML'e göre kodlarken adları birebir kullanın. Metot adları küçük harfle başlar (camelCase).
- Kod Java 25 ile derlenip çalıştırıldı; ekleme, silme ve arama beklenen sonucu verdi.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.